

Towards Exploring Developers' Struggles in Developing Upgradeable Smart Contracts

Zhijie Zhong¹, Jiachi Chen¹, *Member, IEEE*, Jiashui Wang², Qing Xue¹, Jiajing Wu¹, Long Liu¹, Xinlei Ying¹, and Zibin Zheng¹, *Fellow, IEEE*

Abstract—Implementing upgradeable smart contracts (USCs) has become a trend in Decentralized applications. Due to blockchain immutability, ensuring the upgradeability of smart contracts requires specialized implementation strategies. A systematic study of developers' concerns regarding USC development can provide insights to reduce development costs and increase software robustness. In this work, we propose the first empirical study on exploring developers' concerns over USCs. We first extract 2,224 USC-related posts based on an iterative process combining keyword filtering and manual filtering of posts from the *Ethereum StackExchange* community. Following open card-sorting practice, we propose 13 common development concerns based on the extracted posts, as well as the causes of these concerns. Furthermore, we analyze the frequency of these concerns within developer discussions. We highlight the most significant concerns of developers, where the top 5 most frequently discussed concerns are notably absent in existing research, e.g., code bugs originating from interacting and testing USCs in specific development frameworks. Additionally, we examine the real-world impact of these concerns by analyzing on-chain smart contracts and security reports from two widely referred databases, i.e., *Rekt* and *Slowmist*. Based on case studies of the USC-related security reports in the past two years, we found that the causes of most USC security incidents are related to the identified concerns. Besides, we proposed a semi-automatic tool based on static analysis to detect related bugs and found 26 bugs in real-world smart contracts, which have involved over 0.3 million transactions. Based on these findings, we provide suggestions on the less-solved-yet-prevalent concerns regarding usability and security of USC development, such as facilitating the testing on USCs under existing development frameworks.

Index Terms—Upgradeable smart contracts, empirical study.

Received 31 December 2024; revised 16 August 2025; accepted 3 September 2025. Date of publication 18 September 2025; date of current version 14 November 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB2703600, in part by the National Natural Science Foundation of China under Grant 62032025, and in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2025A151011632. Recommended for acceptance by Prof. R. Mirandola. (*Corresponding authors: Jiachi Chen; Jiashui Wang.*)

Zhijie Zhong, Qing Xue, Jiajing Wu, and Zibin Zheng are with the School of Software Engineering, Sun Yat-sen University, Zhuhai 519000, China, and also with Guangdong Engineering Technology Research Center of Blockchain, Sun Yat-sen University, Zhuhai 519000, China (e-mail: zhongzhj3@mail2.sysu.edu.cn; xueq25@mail2.sysu.edu.cn; wujiajing@mail.sysu.edu.cn; zhizbin@mail.sysu.edu.cn).

Jiachi Chen is with the School of Software Engineering, Sun Yat-sen University, Zhuhai 519000, China (e-mail: chenjch86@mail.sysu.edu.cn).

Jiashui Wang is with Zhejiang University, Hangzhou 310000, China (e-mail: 12221251@zju.edu.cn).

Long Liu and Xinlei Ying, Individual Researchers, residing in Hangzhou 310000, China.

Digital Object Identifier 10.1109/TSE.2025.3609077

I. INTRODUCTION

DECENTRALIZED Applications (DApps) are composite programs running on blockchains. Composed of multiple smart contracts, a DApp can distribute complex business logic into individual modules running on different addresses. According to *DappRadar*, more than 15 thousand DApps are running on the blockchain platforms, with more than 4 million daily active user addresses [1]. Among these DApps, smart contract upgradeability has emerged as one of the critical requirements in their maintenance paradigms. The main motivation arises from the necessity for dynamically adapting business logic to evolving market requirements, as well as implementing security patches for vulnerability remediation. Thus, implementing upgradeability becomes a new trend in developing DApps [2], [3], [4]. As of December 2023, 56 out of the top 100 most popular DApps in *DappRadar rankings* [1] have adopted the upgradeability pattern.

However, due to blockchain's immutability, achieving the upgradeability of smart contracts requires specialized strategies. A series of Ethereum Improvement Proposals (EIPs) are proposed to define new implementation standards for upgradeability (*c.f.* Section II). According to the study proposed by Li et al. [3], the proxy-based upgradeability strategies have accounted for over 96% among all upgradeability strategies in practice. The main idea of these strategies is to separate the smart contracts in a DApp into the interface contract (proxy contract) and the implementation contract (logic contract). In this strategy, users can interact with a proxy contract whose contract address will not be changed. The proxy contract will forward the function call to a contract address carrying the business logic implementation, whose address is stored as a state variable. In this way, developers can deploy a new logic contract and update the corresponding variable in the proxy to the new logic contract address. Thereby, the proxy can forward function calls to the new logic contract.

Achieving upgradeability in practice requires specialized knowledge and implementation strategies in the context of blockchains. Developers must deal with specific concerns with upgradeable smart contracts (we refer to these smart contracts as USCs in this paper), such as the interaction strategies between proxy and logic contracts, storage collision, and USC initialization problems. The practical complexity related to implementing upgradeability-specific functionalities can add difficulty to DApp development. Moreover, the flawed

implementation of upgradeability can even cause financial losses to DApps. The risks in upgradeability implementation distinguish from common smart contract vulnerabilities such as reentrancy [5], [6], [7] or oracle manipulation [8], [9], as they originate from the particular design patterns of USC. These risks do not cause direct fund loss like general smart contract vulnerabilities. Instead, they undermine the reliability of smart contracts by damaging smart contract storage [10] or allowing adversaries to take control of the upgradeability privilege [2], [3]. For example, the DApp Audius Finance is reported to have lost \$6 million due to flawed upgradeability implementation in their proxy and the logic smart contracts [11].

Therefore, understanding developers' concerns in USC development can provide guides to reduce development costs and enhance DApp security. Current research mainly focuses on characterizing USC usage [2], [4], [12], [13] and analyzing limited security properties of USC implementations [3], [10]. The most related empirical studies to this topic are ProxyHunt [2], USCDetector [3], and [4]. The main goal of these studies is to effectively detect USCs patterns and analyze the number of deployed USCs over time. ProxyHunt and USCDetector also propose to detect security risks in USCs, such as function signature collision, access control, and storage collision. However, none of the existing works have explored the development concerns for USC from developers' perspectives, which can lead to the discrepancy between academic research and USC development in practice. On the one hand, some of the USC concerns can be overestimated by existing works. For example, the issue of function signature collision has been well studied by ProxyHunt. Yet, there are few real-world security cases related to this issue [2]. Besides, this topic is also rare in developers' discussions. According to our findings, there are only six relevant discussions among all the discussion posts on the Ethereum StackExchange platform. The details will be given in § IV. On the other hand, some of the concerns can be underestimated, such as developers' security concerns for developing customized business logic with third-party USC templates. This topic is frequently discussed on Ethereum StackExchange, but is notably absent in existing studies.

To bridge the gap between USC development practice and academic research, we present the first empirical study to identify the primary concerns of USCs developers. Our study differentiates from existing research in that we study these concerns by comprehensive analysis from the developers' perspective and focus on the entire USC software development lifecycle. Thus, we propose demonstrating the categorization and root causes of developers' concerns that existing studies have not addressed. Our study focuses on the following research questions:

RQ1. What are developers concerned about when developing USCs?

Identifying concerns in USC development can guide the design of solutions to reduce development costs and increase development efficiency. Similar to Venkatesh et al. [14], we define developer concerns as discussion topics drawn from developer forums. These topics can contain the design and

implementation problems, as well as identifying and preventing smart contract bugs. We first extracted 2,224 USC-related posts from *Ethereum.StackExchange*. Following open card-sorting practice on software engineering [15], [16], [17], we leveraged a two-step categorization process to extract common development concerns from the posts. In the first step, two of our researchers collaboratively examined a subset of the collected posts and identified an initial set of discussion topics related to these posts. In the second step, the authors independently categorized the remaining posts and compared their results. Through comparison and discussion of the differences, we examined the quality of our manual analysis and refined our categories to 13 distinct development concerns.

RQ2. What is the distribution of these development concerns?

Assessing the distribution of the concerning issues provides insights into their perceived importance by developers. By analyzing the frequency of related posts, we identify the most discussed concerns, such as testing challenges with respect to specific development frameworks. On the contrary, we also highlight the less-concerning issues for developers, such as the function selector collision issue. Furthermore, we point out the frequently discussed yet unsolved problems in the current USC study. By comparing the identified concerns with problems proposed in previous studies [2], [3], [4], [10], we reveal that 9 out of the 13 concerns have not been studied in existing research.

RQ3. What is the real-world security impact of these concerns?

Security is one of the most concerning topics for smart contracts. We study the impact of the identified concerns on security by investigating security incidents related to USC and scanning real-world smart contracts running on the blockchain. For security incidents, we collect and extract the root causes of recent security reports, analyze their relationship with the identified concerns, and examine how frequently these concerns appear across these incidents. We show that the majority of security incidents can be derived from the identified concerns. For smart contracts, we propose to scan the concerns that can lead to code flaws in real-world smart contracts. By analyzing the real-world smart contracts, we found 26 related bugs, including storage collision, flawed initialization, and flawed access control. These smart contracts have involved over 0.3 million transactions, indicating a strong impact.

The main contributions of this paper are:

- We identified 13 development concerns from developers' discussions, where 9 of them are absent in existing research. These concerns cover three phases of USC software development life cycle: *design & Implementation*, *testing*, and *maintenance*.
- We study the distribution of these concerns among developers' discussions. We show the most concerning problems from the developers' perspective. Moreover, we estimate their impact on real-world smart contracts.
- We provide findings that reveal a set of potential improvements on the usability and security of USC development framework, third-party template, and bug detection tools.

All the code and data is available at <https://github.com/Z-Zhijie/USCstudy>.

II. BACKGROUND

In this section, we briefly introduce how smart contract upgradeability can be achieved.

Smart Contract Call. Smart contracts are programs running on the blockchain platform. Ethereum, one of the most popular blockchain platforms, proposed an execution environment for smart contracts, i.e., the Ethereum Virtual Machine (EVM). It defines a set of operation codes (opcodes) and their state transition rules [18]. The deployment and execution of each smart contract opcode cost gas, a type of financial resource, to drive EVM's execution. Therefore, most smart contracts are designed to be as concise as possible to reduce the execution cost.

During the execution, smart contracts call each other by external function calls. Function calls between smart contracts can be achieved by three opcodes, i.e., CALL, DELEGATECALL, and STATICCALL. If a smart contract is called by the CALL opcode, it will execute the corresponding function using the state variable stored in its own storage and return the execution result to the caller if possible. The DELEGATECALL opcode serves a similar functionality to the CALL opcode, except that the execution of DELEGATECALL is based on the caller's context (i.e., changing the caller's state instead of the callee's state). The STATICCALL is also similar to the other two opcodes, except that there will be no state changes in either of the smart contracts. These call opcodes serve as connectors between individual smart contracts running on the blockchain.

Upgradeable Smart Contracts. To meet the demand for smart contract upgradeability, a series of EIPs are proposed to provide design strategies for achieving upgradeability. Proxy-based strategies are the most popular EIPs designed for USCs, including EIP-897 DelegateProxy [19], EIP-1538 Transparent Contract Standard [20], EIP-1822: Universal Upgradeable Proxy Standard (UUPS) [21], EIP-1967 Proxy Storage Slots [22], and EIP-2535: Diamond Standard [23].

As introduced in Section I, the main idea of these design strategies is to use the proxy-logic strategy. Fig. 1 illustrates the main workflow of this strategy, which consists of two main smart contracts, i.e., the proxy contract and the logic contract. When users want to invoke a business function, they should first send the function call to the proxy contract. Once the proxy contract receives a business function call, it will execute the DELEGATECALL operation to the logic contract. As described before, the DELEGATECALL mainly takes the target contract address and the function call information as parameters. In the context of upgradeable smart contract, the target contract (i.e., logic contract) address is stored in the storage of the proxy contract, and the function call information is received from the user. Since the DELEGATECALL can execute code in the logic contract based on the storage state of the proxy contract, any storage state changes during the execution will be logged in the proxy contract's storage. Developers can upgrade a DApp by deploying a new logic contract and updating the logic address

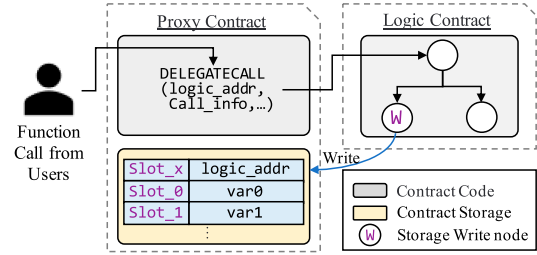


Fig. 1. Workflow of the proxy-logic strategy.

variable. In this way, the interface (i.e., proxy contract address) will not be changed, and the data will not be lost as they are stored in the proxy contract.

III. RQ1: IDENTIFICATION OF DEVELOPMENT CONCERNS ON USC

Understanding developers' demands and concerns has been a major topic in software engineering [24], [25], [26], [27], [28], [29], which can guide the design of tools to facilitate software development, testing, and maintenance in practice. Previous works focus on the detection, characterization, and security of USCs. However, none of them have studied the development concerns for USC. To demonstrate the overall picture of developers' concerns on upgradeability, we conducted an analysis of Ethereum StackExchange posts in this section.

A. Approach

1) Data Collection and Preprocessing: Our study explores developers' concerns about upgradeable smart contract development by collecting and analyzing their available discussions. Since programmers often post discussions and questions on Q&A sites, we can directly reach their discussions through the posts. Following existing studies [16], [30], our study focuses on posts extracted from Ethereum StackExchange, one of the most popular and widely used Q&A sites for Ethereum users. Sources such as GitHub issues and Reddit discussions are not involved in our study, as they exhibit distinct structures compared to discussions in posts.

To balance the time costs and coverage of discussions on upgradeability, we use keyword filtering for the StackExchange posts. The target of our filtering covers all posts on StackExchange proposed before April 2024. For keyword selection, we consider their relevance to the topic of USC and its implementation methods. Initially, we use the terminology "upgradeable smart contract" as a keyword. Additionally, we also include "proxy smart contract", as our study focuses on the proxy-logic implementations of upgradeability. We leveraged the Ethereum Stackexchange API [31] to perform keyword searches and found 273 and 900 results, respectively. In further investigation, we noticed that some of the upgradeability discussions do not necessarily contain the complete keywords. For example, there are discussions where programmers use alternatives or abbreviations such as "upgradeable token" or "proxy" to refer to the USCs. To cover more relevant discussions, we

TABLE I
NUMBER OF POSTS COLLECTED FROM EIP-SPECIFIC KEYWORDS

Keywords	# Collected	# Overlapped
EIP-897 DelegateProxy	3	3
EIP-1538 Transparent Upgradeable Proxy	58	52
EIP-1822 UUPS	46	44
EIP-1967 Proxy Storage Slots	53	51
EIP-2535 Diamonds, Multi-Facet Proxy	24	16
All	148	130

refine our filtering and only keep the words that necessarily relate to upgradeability in the keywords, i.e., “upgradeable” and “proxy”. The filtering results in two extended sets of posts, accounting for 1,303 and 1,237 posts, respectively.

Since the posts are collected by keyword filtering, a potential risk lies in missing important posts. We propose to evaluate the representativeness of the collected posts. Specifically, we conduct another round of keyword filtering using a set of more fine-grained keywords within the same period, i.e., the EIPs related to USCs (as introduced in Section 2) before April 2024. Taking “EIP-1822: Universal Upgradeable Proxy Standard (UUPS)” for example, we extract the keywords from 1) the EIP number: “EIP1822” and “EIP-1822”; 2) the title: “Universal Upgradeable Proxy Standard”; 3) the common used abbreviation: “UUPS”. We then check the overlaps between the results and our original results. The results are shown in Table I. Two observations can be drawn from the table: 1) We collected a total of 148 posts regarding specific upgradeable patterns, which are significantly less than posts regarding general keywords, i.e., “upgradeable” and “proxy”. The difference between the numbers indicates that developers tend to be concerned about more general concepts on USCs rather than specific upgradeable strategies. 2) 130 out of 148 posts can be covered by the results from the posts collected before, showing the representativeness of our keywords.

We further process the collected data regarding two aspects: post duplication and topic relevance.

There are two types of duplicated posts: direct duplication and semantic duplication. Firstly, the search results from the two keywords can overlap, as one post can contain both keywords. For this type, we remove overlapping posts by their links. Secondly, there can be semantically duplicated posts where two individual posts may discuss the same question. For example, the StackExchange will prompt that “*the question already has an answer in another post*” for some of the semantically duplicated posts. We keep these posts in our analysis, as they can reflect the common concerns of different developers.

Collecting posts from keywords also brings irrelevant discussions to our results. We noticed that the search results from “upgradeable” can contain posts discussing irrelevant subjects, such as upgrading the version of blockchain clients, compilers, and development frameworks. The search results from “proxy” can contain posts discussing the use of network proxies. For example, developers may ask “how to correctly set network proxy to communicate with blockchain networks”, which is not relevant to smart contract upgradeability and should be omitted

OZ transparent upgradeable proxy admin reverts when trying upgrade an implementation

Title

I have this transparent upgradeable proxy deployed on sepolia. When I try to call the upgradeAndCall function with the data being 0x it reverts at some point...

Description

OpenZeppelin v5 TransparentUpgradeableProxy creates its own ProxyAdmin contract. Now, in the proxy constructor you should provide not the ProxyAdmin contract address, but let's say the Admin's address of the future ProxyAdmin Contracts.

So the issue you have is that **your admin contract is not the admin of the proxy**, that's why the upgradeAndCall call is forwarded to the implementation.

Comment

Fig. 2. Example of a card of a StackExchange post.

from our results. In this step, all results were manually analyzed by two authors of this paper, each with at least two years of smart contract development experience. For each collected post, both authors reviewed it in a meeting. They then jointly determined whether the post was related to USC by checking if the discussion subject was the upgrade or the upgrade mechanism on smart contracts. As a result, no disagreement occurs, and a total of 781 posts are found to be related to upgradeability.

2) *Open Card Sorting*: We label the topic of the collected posts following the open card sorting approach [15]. Open card sorting is a method to organize information into non-predefined categories, which has been widely used in many studies for categorizing topics from developers' discussions or interviews [16], [32], [33]. The idea is to formalize related information into cards for each item and assign it to logical categories iteratively. Specifically, we create a card for each Stack Exchange post, including related information such as the post title, description, and comments.

Fig. 2 shows a simplified example of a card. The card consists of three types of information, i.e., the post title, the description, and the comments. From the title, we can determine that the post describes a problem while using the access control functionality of OpenZeppelin's (i.e., OZ) transparent upgradeable proxy contract template. The comment indicates the root cause is that the new version of OpenZeppelin's “TransparentUpgradeableProxy” contract template updates the access control strategy. The new access control strategy defines the contract administrator address as a newly-deployed “ProxyAdmin” contract instead of the address provided by the developer. Therefore, we infer the categories for this card are “Third-party USC template Usage” and “Flawed Access Control”.

In the open card sorting process, we follow the detailed steps as in [16] to determine the categories of these cards.

Step 1. We randomly choose 20% of the card items to discuss their categories. For each card item, two of our authors separately read the post title, question description, and comments to decide what topic the programmer is concerned about. Then, the two authors discuss together to see if they agree on the concerns and categorize them based on the context. As shown in the above example, we form categories based on the root concerns of the posts, i.e., what caused the developer to post the question on Ethereum StackExchange. Since a post can be

TABLE II
CATEGORIZATION OF DEVELOPERS' CONCERNS IN IMPLEMENTING UPGRADEABILITY

Concern	Description	Is Studied?		
		[2]	[3]	[10]
USC Workflow Understanding	Unfamiliar with the workflow of upgradeable strategies	✗	✗	✗
Third-party USC Template Usage	Lack of knowledge on inheriting customized function with third-party template.	✗	✗	✗
Proxy-Logic Interaction	Hard to design encoding scheme in the low-level proxy-logic contract call.	✗	✗	✗
USC with Development Framework	Difficult to interact and test USCs in development framework.	✗	✗	✗
Implementation with Less-used Language	Lack of tutorials on developing USCs with less-popular program language.	✗	✗	✗
Gas Estimation of USC pattern	Hard to estimate gas cost after implementing USCs.	✗	✗	✗
Code Size Exceeds Maximum Limit	Lack of tutorials to split USCs into several sub-contracts.	✗	✗	✗
Trust Concern on Upgradeability	The upgradeability can lead to centralization problem.	✗	✗	✗
Storage Collision	Storage layout between proxy and logic contract conflict with each other.	✓	✗	✓
Flawed Initialization	Ineffective storage variable initialization in the logic contract.	✗	✗	✗
Flawed Access Control	The access control of upgradeability can be ruined or bypassed.	⊖	⊖	✗
Function Selector Collision	The functions of proxy and logic contract use the same selector hash.	✓	✗	✗
USC Runtime Status Tracing	Hard to trace the proxy/logic contract states during maintenance.	⊖	✓	✗

motivated by concerns on multiple topics, the authors can label the post into multiple categories. The categories are formed dynamically as we iterate through the cards. If any change is made to a category during card sorting, such as merging, the authors will recheck the posts that have already been labeled in that category to ensure the modification is valid for all posts belonging to it. If there is a discrepancy on any of the categories of a post, we will invite an expert with more than six years of smart contract development. The three authors will discuss which topics the post belongs to. If the root concern was unclear or they cannot reach a consensus on the category, we omitted it from our card sort. As a result, we come across 12 categories of developers' concerned problems in upgradeable smart contract development.

Step 2. The two authors individually read the remaining 80% card items and labeled them into the above categories. During this step, we noticed several posts with similar topics, but these topics do not belong to the above categories. By reading the descriptions, the authors agreed that these posts are related to a new category, i.e., "Implementing with Less-used Languages". Finally, we categorized 13 concerned problems from the developers' perspective, as shown in Table II. To measure the agreement between the two authors, we utilized Cohen's Kappa value [34] based on the categorization results. The overall Kappa value is 0.84, showing strong agreement and stability of our categorization.

During the card sorting process, the categories are dynamically formed with the consent of both authors.

A category merge is made with the category "USC Workflow Understanding" in the first iteration. Specifically, there are posts discussing the conceptual understanding of USCs. Depending on the topics of these discussions, we labeled them into a set of categories, including understanding the implementation of upgradeability, concerns caused by misunderstanding upgradeability, etc. However, during the iteration, we found that these topics share the same root cause, as they are all caused by developers' lack of conceptual understanding of the USC workflow. Therefore, we merged these posts into the category "USC workflow understanding", indicating the discussions in these posts are related to the same concern.

A new category "Implementation with Less-used Language" is added during the second iteration. The authors noticed that there are posts discussing how to develop USC with minority program languages such as Vyper. These discussions are caused by the lack of guidance on USC development with less popular languages, which is distinct from the concerns identified in the first iteration. The number of this type of post is very low, and none of them occur in the posts selected in the first iteration. Therefore, we add a new category for this type of concern. Notably, we only add a category with the consent of both authors. For example, post 82919 [35] asks about verifying the source code of a proxy contract on Etherscan. One of our authors holds the opinion that the post belongs to a new category about using Etherscan services for USCs. The other author thinks that this post should be unrelated to USC concerns. After our discussion, the authors reached an agreement that there is no difference between verifying a proxy smart contract and verifying a normal smart contract. Therefore, the post is labeled as unrelated.

B. Results

As shown in Table II, we find 13 developers' concerns in developing USCs. We also note in the table whether they have been studied in existing literature [2], [3], [10]. We did not integrate [4] in the comparison. Because the main goal of their study is to understand the prevalence, types, and use cases of proxy contracts, rather than development concerns or code bugs. Notably, these concerns occur in different stages of the software development lifecycle, whereas existing research mainly focuses on the testing and analysis stage. Addressing the requirements in the Software Development Lifecycle has been a research topic in traditional software programs [36]. Therefore, we further divide these concerns into three main phases according to the stage when these concerns occur, i.e., *design & implementation*, *testing*, and *maintenance*. In the following part, we explain these concerns and related studies in detail.

1) *Concerns in Design & Implementation Phase:* In this phase, developers decide how the USC will be structured and implement it in specific smart contract code. We found that

related development concerns mainly fall into the difficulties developers encounter in practical understanding and implementing USC functionalities.

C1-1 USC Workflow Understanding. A primary concern of developers lies in understanding the workflow of upgradeability strategies. Less experienced developers are not familiar with the execution order of upgradeable smart contracts, as well as the communication pattern between proxy contracts and logic contracts. One typical question concerns the pros and cons of different USC strategies [37].

C1-2 Third-party USC Template Usage. Developers can encounter practical difficulties in programming USCs. Currently, many third-party implementation templates are provided by blockchain companies such as OpenZeppelin. The difficulty of inheriting or customizing functions based on these third-party templates formalized the template usage concern. These templates usually involve additional functions or specialized mechanisms to ensure a set of security properties of USCs. For example, the Transparent Proxy template provided by OpenZeppelin is integrated with the access control to ensure the proxy contract cannot be modified by unauthorized users. There are cases in which developers misunderstand the template-provided mechanisms or these mechanisms conflict with developers' customized functions, leading to programming bugs as discussed in post 109323 [38].

C1-3 Proxy-Logic Interaction. Concerns also occur when developers try to program customized functionalities under the context of upgradeability. As introduced in Section II, the interaction between proxy and logic contracts is encoded in low-level inline assembly code. Specifically, the function call to the logic contract is packed and forwarded by the `DELEGATECALL` operation. During the interaction, developers may not be familiar with the encoding scheme of the low-level call, especially when the function parameter types are complex. For example, post 66276 [39] asks for tutorials on calling the logic contract with complex parameters such as dynamic array.

C1-4 USC with Development Framework. During the programming of smart contract code, developers usually rely on development frameworks to test program properties. Several development frameworks are available for smart contract programming, such as Hardhat [40], Foundry [41], Truffle [42], and Brownie [43]. Under these frameworks, developers need to encode customized transactions if they need to interact with the smart contracts. Due to the proxy-logic nature of USCs, the business logic functions are usually encoded in the logic contracts, while the interfaces of these functions are encoded in the proxy contracts. Therefore, how to encode a blockchain transaction to initialize, interact, and maintain the USCs under these frameworks poses specialized obstacles to developers.

Example. Fig. 3 illustrates a case in calling the `foo()` function in the logic contract under Brownie development framework [43], which is discussed in post 100223 [44]. Since the function is forwarded by the proxy contract defined in line 1, there is a specific encoding scheme for the framework to call the function (lines 5-6). However, there are cases where developers encounter errors when using the wrong encoding scheme, such as in line 3.

```
1 proxy = TransparentUpgradeableProxy[-1]
2 // wrong call to foo() in the logic contract
3 print(proxy.foo())
4 // correct call to foo() in the logic contract
5 proxy_logic = Contract.from_abi("Logic", proxy.
6     address, Logic.abi)
7 print(proxy_logic.foo())
```

Fig. 3. Code example of concerns about developing USC under specific development framework.

C1-5 Gas Estimation of USC pattern. A common development concern falls into estimating additional costs introduced by upgradeability. Implementing the upgradeability costs additional gas in two aspects: *code deployment* and *code execution*. As for deployment, developers must write additional code to ensure upgradeability. Consequently, additional gas fees are required to deploy each opcode on the blockchain. As for execution, the call to the logic contract is forwarded by the proxy contract, thereby introducing additional operations in the execution. Each executed opcode also costs additional gas. Since it is not easy to measure the additional deployed and executed opcode, developers face the difficulty of estimating additional gas costs during development as discussed in [45].

C1-6 Code Size Exceeds Maximum Limit. The size of a smart contract can exceed the maximum code size or maximum gas cost per block on Ethereum. While implementing Diamond pattern USC can be a solution, developers may ask how to reduce the code size through USC implementation. For example, post 147064 [46] asks for advice on reducing the size of single smart contracts.

C1-7 Implementation with Less-used Language. Code reuse from open-source repositories is important in modern software development [47], [48]. Currently, most of the third-party templates of USCs are written in Solidity, the most popular smart contract programming language. When programmers are developing with less popular programming languages such as Vyper, there are few code samples that they can refer to, as discussed in post 149380 [49].

C1-8 Trust Concern on Upgradeability. In the design phase of USCs, trustworthiness is also a point of concern in discussions among developers and users. In most implementations, developers can upgrade the logic contract to a new contract without restriction. In such cases, malicious developers can manipulate or ruin users' data stored in the smart contract without their permission. The problem is even worse when developers deploy a benign smart contract to earn users' trust and upgrade it to a malicious one. Therefore, there are discussions such as in [50] about methods to restrict developers' power on upgradeability based on specialized contract design strategies.

2) Concerns in Testing Phase: The main goal of testing phase is to verify that USC functions as expected. According to our investigation, discussions in this phase mainly focus on the following four USC-specific vulnerabilities and defects.

C2-1 Storage Collision. Storage collision originates from the upgradeability design pattern, i.e., separating contract storage and contract logic. The opcode `DELEGATECALL` allows a smart contract to execute code from another contract with its

```

1contract Proxy {
2    address logic_addr;
3    function burn(uint256) {...}
4    ...}
5contract Logic {
6    uint256 version;
7    function collate_propagate_storage(bytes16)
8        {...}
9    ...}

```

Fig. 4. Code example of storage collision.

own storage. By using DELEGATECALL in the proxy contract, the logic contract can change the storage of the proxy contract. As a result, if the proxy contract and logic contract use the same storage slot with different proposes, the value stored in that slot can be damaged. To detect this issue, USCHunt [2] proposed a cross-contract analysis method based on source code, where 60 issues were detected. [10] proposed a bytecode-based collision detection method based on type inference and life-span analysis, as well as exploit generation for the collision.

Example. Fig. 4 illustrates an example of this defect. The proxy contract contains an address type variable named `logic_addr` as the first storage variable. According to EVM implementation, the first variable is stored in the first storage slot (i.e., `slot_0`) of the smart contract. Similarly, the logic contract also defines a `uint256` type variable `version` in storage `slot_0`. The problem occurs when the proxy contract DELEGATECALL the logic contract to modify the `version` variable using the proxy contract's storage. Because the operation on the `version` variable changes the value in storage `slot_0`, which is storing the value of `logic_addr`. In this case, the storage collision occurs, and the value of the `logic_addr` is damaged. Storage collision can cause serious damage. For example, Audius Finance [11] was reported to lose \$6 million due to the storage collision defect.

C2-2 Flawed Initialization. As introduced in the storage collision concern, the storage access between proxy and logic contracts can confuse developers. The flawed initialization is also caused by misunderstanding the storage access pattern. Developers can falsely set the storage in the wrong contract when initializing USCs.

Example. As shown in Fig. 5, the logic contract defines an address variable `proxyOwner` in line 1. Flaws could occur when the developer initializes the variable in the contract constructor (line 5), a common initialization approach in smart contract programming. The execution of USCs is based on the storage of the proxy contract through DELEGATECALL. In contrast, the constructor of the logic contract sets the value in the storage of the logic contract itself. Thus, the storage value in the logic contract is ineffective when the user interacts with the proxy contract. Instead, the correct implementation should be achieved by calling the proxy contract to forward (i.e., DELEGATECALL) the `init` function call in line 8 of the logic contract. In this case, the variable is set in the proxy contract's storage and is effective in the contract execution.

C2-3 Flawed Access Control. Another main code defect is the flawed access control of USCs, where unprivileged users can access sensitive resources in the contract, such as performing

```

1contract Logic {
2    address public proxyOwner;
3    uint keyVar;
4    // wrong initialization
5    constructor(address _owner) public {
6        proxyOwner = _owner;}
7    // correct initialization
8    function init(address _owner) external {
9        proxyOwner = _owner;}
10   // sensitive function
11   function setVar(uint _newVar) public {
12       require(msg.sender == proxyOwner);
13       keyVar = _newVar;}
14}

```

Fig. 5. Code example of flawed initialization and flawed access control.

unauthorized upgrades or writing storage variables. The access control problem has been a widely studied topic for smart contracts [51], [52], [53], [54]. However, access control in USCs can be undermined due to USC-specific issues such as unintended variable assignments (such as in cases of storage collision and initialization) or adversarial attacks against upgradeability. USCHunt [2] and USCDetector [3] partially studied the access control in USCs. USCHunt proposed methods to analyze USC source code and check whether the upgradeability can be removed unexpectedly. [3] proposed a semi-automatic tool to perform restrictive checks on the upgradeability of USCs that have been observed to be upgraded by different owners in history.

Example. As shown in line 11-13 of Fig. 5, the smart contract only allows the user at the `proxyOwner` address to modify the sensitive variable `keyVar`. The access control is achieved by adding the “`require(msg.sender==proxyOwner)`” statement before the variable assignment, where the `msg.sender` refers to the contract caller and the `proxyOwner` refers to the storage variable defined in line 2. This statement checks if the contract caller passes access control (i.e., the value `msg.sender==proxyOwner` is True). Therefore, the value of the storage variable `proxyOwner` is crucial in the access control check. However, developers can set the access control variable to a false value for several reasons, such as flawed initialization or storage collision. For example, if flawed initialization occurs in the example code, only the `proxyOwner` variable in the logic contract is set, and the variable in the proxy contract remains unset. Consequently, the developer cannot pass the access control in line 12 while interacting with the proxy contract.

C2-4 Function Selector Collision. The function selector collision concern originates from the contract calls mechanism in EVM. According to the Ethereum yellow paper [18], the target of a function call to a smart contract is defined by two parts: contract address and target function selector hash, where the latter specifies which function in the smart contract is called. However, the function selector hash is calculated by the first 4 bytes of the hash of the function signature (i.e., function name and parameter types). Since the calculation only takes the first 4 bytes of the hash result, the collision-resistance property is undermined, and two distinct functions might have the same signature hash. USCHunt [2] proposed to detect this issue by

comparing the function selectors between smart contracts. Yet no real-world examples were detected.

Example. As shown in Fig. 4, the proxy contract implements the `burn(uint256)` function, while the logic contract implements the `collate_propagate_storage(bytes16)` function. However, the two functions have the same selector hash as `0x42966c68`. When a user calls the function in the logic contract, only the `burn(uint256)` function in the proxy contract will be executed.

3) *Concerns in Maintenance Phase:* The maintenance phase provides continuous support after smart contract deployment. Among the analyzed StackExchange posts, discussions mainly concern the challenge of tracking the runtime status of deployed USCs.

C3-1 USC Runtime Status Tracing. Since the proxy contract's storage slot to store the logic contract address can be customized in some standards, identifying the corresponding logic contract address / historical logic contract addresses can be challenging for developers. In our investigation, a typical concern from developers is how to trace the proxy or logic contract address given the other one, such as in discussion [55]. USCHunt [2] detects the proxy and logic contract based on a set of features extracted from smart contract source code. [3] further extends the detection and supports recognition of historical contract versions by analyzing contract transactions.

In conclusion, the identified concerns mainly originate from the specialized design of implementing upgradeability, such as the proxy-logic contract interaction, storage-logic separation, etc.

Findings 1: We identified 13 types of developers' concerns and their root causes. Most of these concerns fit into the *design & implementation* and the *testing* phase of USC software development life cycle.

The identified concerns exhibit differences in comparison with existing smart contract vulnerability categorizations [8], [16], [56], [57] based on two reasons. 1) Existing studies mainly focus on the bug categorizations in smart contracts. Instead, most of our concerns (e.g., C1-1 to C1-4, C1-8, and C3-1) reflect developers' concerns during the design and development of USCs. Certain concerns, such as gas estimation (C1-6) and code size limit (C1-7), can be regarded as subproblem of general smart contract problems, which has been studied in [58], [59], [60]. However, extending such studies to USC-specific scenarios requires additional effort. 2) As for the concerns that could lead to code flaws, i.e., C2-1 to C2-4, existing bug categories on the general smart contract study only cover the "flawed access control" [51], [54], [61]. Because the other three concerns originate from the specialized mechanisms for implementing upgradeability. Only studies focusing on USCs [2], [3], [10] partially cover the "Storage Collision" and "Function Selector Collision" concerns.

IV. RQ2: DISTRIBUTION OF THE DEVELOPMENT CONCERNS

In this section, we study the distribution and trend of the identified concerns by calculating the number of posts where these

concerns occur. Moreover, we identify frequently discussed yet unsolved problems by comparing them with existing research.

A. Data Processing

The distribution of these concerns among developers' discussions can reflect their importance. Fig. 6 shows the number of posts in each category. We rank these concerns by their occurrence and mark those that are covered by existing studies. The sum of all the occurrences is greater than the number of all posts, as one post discussion can involve multiple problems developers encounter. For example, an access control vulnerability can be caused by flawed initialization strategies in USCs. In such cases, we label the corresponding post with both categories.

B. Results

As shown in the figure, workflow concerns are the most frequently discussed topic among developers, accounting for 309 posts. The phenomenon originates from the functionality of the Q&A site, where programmers ask questions when they are not familiar with them. Therefore, it is possible that there are more less-experienced programmers than sophisticated programmers asking questions on StackExchange.

Below the primary conceptual workflow of USCs, there are the concerns that developers encounter during implementation and testing, i.e., the Proxy-Logic Interaction, Third-party USC Template Usage, USC with Development Framework, Flawed Initialization, and Storage Collision. All of these concerns occur more than 60 times in the StackExchange discussions. The first three concerns are questions about how to implement a specific functionality during upgradeable smart contract development. In such discussions, developers usually ask for a guide for specific code implementation, saving development efforts. The latter two concerns occur when developers encounter unexpected program execution results in the testing phase. These discussions usually consist of the smart contract code, the expected execution result, and the actual execution result.

The flawed access control and runtime status tracing for upgradeability are relatively less popular in the discussions, accounting for 33 and 25 times. Possible explanations could be that the runtime status tracing is at post-deployment stages, whereas not all the USCs in development will be finally deployed on the blockchain. In contrast, the relatively low frequency of access control concerns is more surprising given its critical importance. One plausible explanation is that access control issues are more critical in production-grade deployments, where developers may rely more heavily on internal audits rather than public forums. The remaining five topics are rare in the developers' discussions. Among the 781 upgradeability-related posts, their occurrences account for less than or equal to 20 times. A possible explanation is that developers rarely encounter such situations. For example, there are few discussions on the "Function Selector Collision", since two functions rarely have the same selector (i.e., the first 4 bytes of hash of the function signature) because of the collision resistance of hash functions.

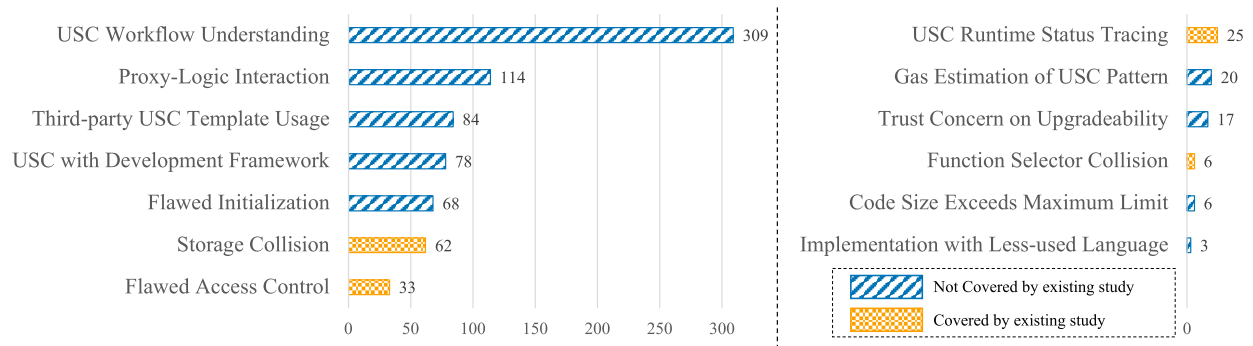


Fig. 6. Distribution of the identified concerns.

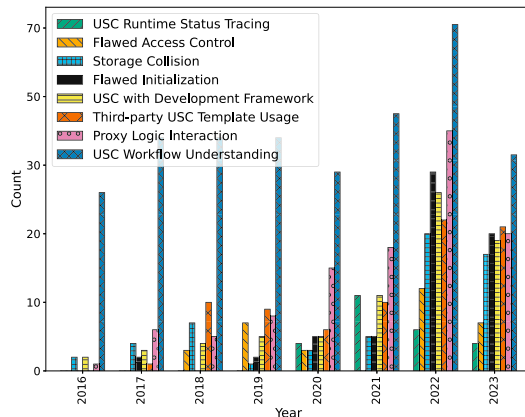


Fig. 7. Distribution of each concern by year.

Findings 2: The majority of the discussions are distributed in the design & implementation phase, while initialization and storage collision are also hot topics in the discussions.

We further studied the trend of these concerns in recent years. As shown in Fig. 7, we show the number of posts on each concern by the year they are proposed. We excluded concerns that account for less than 20 times as few samples may not reflect the trend. We also exclude posts in 2024 as our data is collected till April 2024. As illustrated in the figure, the discussions on all of the concerns have been increasing over the years. While there have been consistent discussions on the majority of the concerns from 2017, the concern on runtime status tracing newly emerged in 2020. This trend aligns with the increasing number of proxy-based USCs deployed on Ethereum, which has grown significantly since 2020 [2] and increased the demand for tracing the on-chain status of deployed USCs. Besides, the concern about flawed initialization was rarely discussed before 2020 but has gained increasing attention in recent years. The increase might be caused by the introduction of OpenZeppelin's initializable characteristic in their USC template [62].

Findings 3: The discussions on these concerns have been increasing over the years, while concerns on runtime status tracing and flawed initialization newly emerged in recent years.

In this section, we also studied the relationship between the identified concerns and existing research works. We mark the concerns covered by existing studies as blue bars in Fig. 6. As shown in the figure, the 6th, 7th, 8th, and 12th concerns have been previously studied. These concerns are distributed in the testing and maintenance phases of USCs. The Storage Collision, Access Control, and Runtime Status Tracing are relatively prevalent in the discussions, all accounting for over 20 times. However, the Function Selector Collision concern is only discussed 6 times out of the 781 discussions, showing the gap between existing research and practical obstacles in development. Moreover, there are discussions showing that the risk of Function Selector Collision may be very low from the developers' perspective by referring to audit reports proposed by OpenZeppelin. For example, in OpenZeppelin's audit report for the DApp "dYdX Perpetual" [63], the report suggests that the DApp should add a restrictive function to avoid the function selector collision problem in the DApp. However, the developers did not take the advice because the problem can be easily tested during development.

We also observed that 9 out of the 13 difficulties have not been covered by existing studies. Moreover, none of the top 5 prevalent concerns have been studied. The 2nd-4th concerns are all practical problems at the implementation phase, while the Flawed Initialization can be detected via testing. Given the prevalence of these uncovered concerns, developing tools for them can increase development efficiency and reduce software flaws for a large scope of upgradeable smart contract developers.

Findings 4: Among the 13 identified concerns, only 4 of them have been previously studied. The top 5 most popular concerns are still not covered by current USC research.

V. RQ3: REAL-WORLD SECURITY IMPACT OF THE DEVELOPMENT CONCERNS

In this section, we study the real-world security impact of these concerns by analyzing security reports and real-world smart contracts.

A. Perspective From the Security Incidents

1) *Data Collection and Processing*: Security incidents can reflect the impact of related concerns on real-world projects. Thus, we extend our study on StackExchange discussion analysis to related security incidents. Following existing empirical studies on smart contract [64] and decentralized application [8], [56], we collect security incidents from *Rekt.news* [65] and *Slowmist hacked database* [66]. These two databases provide comprehensive blockchain security reports and analyses of the root causes. The incidents we collected cover a period of over two years, from January 2022 to June 2024. Similarly to our analysis in RQ1, we filter the potential reports related to upgradeability by the keywords “upgrade” and “proxy”. Through the filtering, 14 reports are collected.

Since upgradeability is possibly not the root cause of these reports, we performed a manual check on their relevance to upgradeability. This filtering process is similar to the process used for excluding irrelevant posts in RQ1. Specifically, the same two authors who categorized concerns in RQ1 checked the root causes and attack strategies described in the reports. Next, they independently determined whether an incident was related to upgradeability. The authors then compared their results. An incident is labeled as USC-related if it is caused by smart contract upgrades or upgrade mechanisms, such as the proxy-logic interaction. The security reports typically contain sections providing clear root causes and detailed explanations of the incidents. For example, the Audius Incident report [11] contains a section stating that “*This caused a collision with OpenZeppelin’s Initializable contract ...*”. We refer to the corresponding section to extract the root causes of these incidents. The authors did not encounter any discrepancies when labeling the incident reports. As a result, we collect 11 reports as shown in Table III. The table lists the time, financial loss, project, and descriptions of the recent security incidents related to upgradeability. From the distribution of the incidents, we found an increase in both the number and financial loss of related issues in the last year. Most of the incidents have caused severe financial losses of more than \$1 million, except that the Compound incident caused the related functionality to shut down for seven days.

2) *Analysis Results*: By analyzing the causes of these incidents, we find that most of the incidents are related to the concerns introduced in Section III. Their causes can be divided into low-level mechanism issues and high-level logic issues.¹ Incidents caused by low-level mechanism issues include the Audius Finance hack, the Perpy Finance hack, and the OKX DEX hack. Specifically, the Audius Finance hack is caused by the storage collision problem between its proxy contract and logic contract, which is exactly the issue we found in developers’ discussions. Besides, the Perpy hack originates from the flawed initialization of their proxy contract. The OKX DEX incident reflects the trust concern of USCs. In this incident,

¹We refer to the definition of high-level logic issue in [52], where a logic issue refers to a flaw arising from high-level security policy design and enforcement. Instead, a low-level mechanism issue refers to flaws that follow common vulnerability patterns and can be avoided by adopting the suggested coding practices.

TABLE III
SECURITY INCIDENTS RELATED TO UPGRADEABILITY

Date	Loss	Project	Description
2022/7	\$ 1M	Audius [11]	There is storage collision between the proxy and logic contracts
2022/8	-	Compound [67]	The upgrade is not compatible with the old functions
2023/2	\$ 1.4M	Dexible [68]	The upgrade undermines the previous access control policy
2023/3	\$ 8.9M	Safemoon [69]	The upgrade undermines the previous access control policy
2023/4	\$ 6.5M	Deus DAO [70]	The upgrade undermines the previous access control policy
2023/5	\$ 1.1M	Level-Fi [71]	The new function in the upgrade is not compatible with the old functions
2023/6	-	Astaria [72]	Attacker could invoke the self-destruct feature through the proxy contract.
2023/12	\$ 2.7M	OKX DEX [73]	The proxy owner upgrades the logic smart contract to an unsafe version
2024/3	\$ 2M	Unizen [74]	The upgrade undermines the previous access control policy
2024/4	\$ 0.2M	NGFS [75]	The smart contract has flawed access control over the upgrade mechanism
2024/5	\$ 0.1M	Perpy [76]	An error occurs in initializing the DeFi’s proxy contract

```
function getUnderlyingPrice(address cToken) {
2 // before upgrade
3   TokenConfig memory config =
      getTokenConfigByUnderlying(cToken);
4 // after upgrade
5   TokenConfig memory config =
      getTokenConfigByUnderlying(CErc20(cToken).
        underlying());
6 ...}
```

Fig. 8. Code example of compound finance upgrade issue.

the proxy owner address can upgrade the logic contract without limitation. Consequently, the proxy owner address upgraded the logic contract to an unsafe version for multiple times, allowing adversaries to steal user assets.

Besides the storage collision and trust concern, six incidents are caused by flawed access control. Notably, the access control issues in four incidents are high-level logic issues beyond the access control of the low-level upgrade functions in the proxy contracts. These issues happen due to inconsistent access control policies between different logic contract versions. In these cases, adversaries can find a path to access specific sensitive resources that should not be accessible before the upgrade.

Moreover, we notice that the cause of the Compound incident belongs to none of the identified concerns in Section III. It is caused by the incompatibility between different logic contract versions. As shown in Fig. 8, the upgrade introduced a new function call `underlying()` to a Compound token interface `CErc20` (line 5). However, this new function call is not compatible with the existing implementations of other contracts in the DApp, as one of the Compound token contracts does not implement the `underlying()` function, leading to an unexpected error during the contract execution. Unlike the OKX DEX incident, the upgrade period of the Compound project is strictly limited. Even though developers noticed the issue shortly after the upgrade, it was merely fixed after 7 days. Consequently, the related module is unavailable for 7 days.

Findings 5: While the causes of most USC security incidents are related to the identified concerns, a new type of issue, i.e., compatibility between upgrades, is discovered by analyzing the security incidents.

The analysis results from the security reports suggest future USC research directions. Among the 11 incidents, only the storage collision in the Audius Finance incident is fully covered by existing research. There are studies concerning the access control of the upgradeability mechanism, but research on the high-level access control policy for business logic between different contract versions is still missing. Besides, the compatibility issue reflected in the Compound incident shows a new categorization of development challenges in USC development, which is complementary to the concerns we find in the StackExchange analysis. Moreover, the OKX DEX incident and Compound incident also reflect the need for a fine-grained trust mechanism for upgradeability. While the loose limitation of upgradeability can easily lead to a fragile trust system, putting strong limitations on upgradeability also hinders developers from efficiently fixing known code flaws.

Notably, although there have been extensive works on detecting logical vulnerabilities in general smart contract study [9], [52], [77], [78], [79], [80], there is still a gap between existing research and practical detection methods for the identified concerns and listed incidents. Some of the works detect the logical flaws based on checking a set of predefined security properties [9], [77], [79], which may not fully cover the specialized vulnerability patterns regarding upgradeability. For example, GPTScan [77] detects logical vulnerabilities based on 12 common logic vulnerability types. However, the bugs, such as the compatibility between contract versions in the Compound and Level-Fi incidents, are not supported. Other works detect logic vulnerability by detecting inconsistencies between contract code and the expected policies, which are extracted from external sources, including web interface [78], [80] and transaction records [52]. Thus, the scope of these works is constrained by the policy sources they rely on.

Findings 6: While existing USC research focuses on low-level mechanism security properties, the majority of security incidents are caused by logical development challenges.

B. Perspective From Real-World Smart Contracts

1) *Data Collection:* In this section we propose to investigate the security impact of these concerns among smart contracts in the wild. We focus on the three concerns in the testing phase: Storage collision, access control, and initialization issue. Our investigation starts with the proxy contract dataset proposed by ProxyHunter [2], where 768 proxy contracts are present in the dataset. There are cases that a proxy contract has no corresponding logic contracts, or has multiple corresponding logic contracts due to several upgrades. Based on the proxy-logic contract labels provided by Etherscan, we extracted 505 proxy contracts with 565 corresponding logic contracts.

2) *Analysis Methods and Results:* In this part, we introduce the method we used to recognize the three issues and the results.

Storage Collision. Storage collision occurs when the code in proxy contract and logic contract uses the same storage slot for different variables. To detect such issues, we first extract the storage slots that are used for write operations (i.e., `SSTORE` in EVM) in both the proxy and logic contracts. Then, we manually check whether the corresponding storage variables in both smart contracts are the same by Ethereum storage querying service providers such as `evm.storage` and `evm-storage.codes`.

Access Control. Access control issues can occur in two cases: 1) the upgrade function is publicly available and misses access control; 2) the access control of the upgrade function can be ruined accidentally or incidentally. To detect such issues, we first find the storage slot used for storing the logic contract address by locating the storage variable used for the `DELEGATECALL` operation in the fallback function. Secondly, we scan through the proxy contract's code to find the storage write operations (i.e., `SSTORE`) related to the target slot, which should be a privileged operation that needs access control. Furthermore, we detect the missing access control issue and ruined access control issue based on `AChecker` [51] and `PrettySmart` [54]. Specifically, we check whether the privileged `SSTORE` operation is protected by a "`require`" operation such as the code "`require(msg.sender==proxyOwner)`" in line 15 in Fig. 5, where the statement checks if the caller address has a privileged role. If the privileged operation is not protected, anyone can manipulate the logic contract address storage variable. Even if the smart contract implements access control, we also note that the access control variable (e.g., "`proxyOwner`") can be ruined due to the storage collision problem. Therefore, we further detect the ruined access control issues by comparing the access control variable to the variables affected by storage collision problem (if exists).

Initialization Issue. As discussed in Section III, the initialization issue is caused by unintended storage write in the logic contract through the constructor. We note there are cases where developers initialize the storage variables in both proxy and logic contracts. To avoid such cases, the main idea is to filter the storage variables that are only initialized in the logic contract's constructor but not initialized in the proxy contract's constructor. The Ethereum smart contract bytecode consists of two parts: the creation bytecode and the runtime bytecode. The former corresponds to the operations run in the constructor, and the latter corresponds to the operations run in the runtime. Thus, we first find the storage slots of `SSTORE` operations that are only used in the logic contract's constructor (i.e., creation bytecode). Secondly, we check whether the same slot is used for `SLOAD` operation in the runtime bytecode. If a storage variable is only initialized in the logic contract's constructor and used in runtime, there can be risks that the initialization is ineffective.

We manually checked the detected results and found 19 proxy-logic contract pairs with the storage collision issue, 6 with access control issue, and 1 with the initialization issue, as shown in Table IV. There were also 19 false positives found in the detection results of access control issue. The main reason is that the tool we used to identify the access control statement failed to correctly recognize all the access control conditions on the upgrade mechanism. No false positive is found in the storage

TABLE IV
REAL-WORLD USC ISSUES

Issue	Storage Collision	Access Control	Initialization Issue
#Detected	19	6	1
#inactive SCs	10	4	0
#active SCs	9	2	1
#Transactions	268,041	2,503	37,362

collision and initialization issue. We further extracted the transactions involved in these smart contracts through *etherscan.io*. The results are shown in the third row of the table. While some of the smart contracts are less active and involve less than 20 transactions, there are highly active smart contracts, such as the vulnerable smart contracts in the Audius Finance incident. These smart contracts involve over 0.3 million transactions, indicating a strong real-world impact.

Findings 7: The identified concerns can also lead to smart contract bugs in practice. We found 26 related bugs in real-world smart contracts, which involved more than 0.3 million transactions, indicating a strong impact.

VI. DISCUSSION

A. Perspectives From USC Developers

Preference on USC patterns. Developers' preferences on specific USC patterns can differ on the USC-related EIPs. The preference can be reflected by the number of posts targeting these USC patterns. We inherited Table I from Section III-A to study the number of posts that contain keywords relating to specific USC strategies. Based on the occurrence of related posts, we find that developers' concerns mainly fall into EIP-1538 Transparent Contract Standard, EIP-1822 Universal Upgradeable Proxy Standard (UUPS), and EIP-1967 Proxy Storage Slots. The early proposed EIP-897 only occurs 3 times in the search results, while the recently proposed EIP, i.e., EIP-2535, both account for less than 20% of all the related posts.

Findings 8: Despite EIPs on implementing USC are continuously updating, developers still prefer early-proposed implementation strategies.

Preference on development framework. We conducted another two statistics on the occurrences of different development frameworks in USC-specific and general posts. As shown in Table V, Hardhat and Truffle have been the most popular frameworks among USC developers, despite that they account for the 2nd and 3rd place in all posts on StackExchange. Besides, although Truffle is popular, it is no longer actively maintained [42], as well as Brownie [43]. The posts concerning the inactive frameworks can provide less guidance for smart contract development in the future. In addition, while Remix is the most discussed framework considering all topics, it is rarely discussed in USC-specific posts. One possible reason is that Remix is a GUI-based framework and does not originally support upgradeable smart contracts before its update in July 2022 [81].

TABLE V
NUMBER OF POSTS CONCERNING SPECIFIC DEVELOPMENT FRAMEWORKS

Framework	Active	# posts (USC-specific)	# posts (All)
Hardhat	✓	34	1428
Truffle	✗	22	2925
Remix	✓	12	3300
Foundry	✓	6	846
Brownie	✗	4	372

B. Enhancing USC Research & Development

The results of our study indicate the demand for improving the usability and security of USC development and research.

Usability. The usability of USC templates and limited support from development frameworks have been a major concern in developers' discussions. While the templates are frequently updated, they lack general guidance on safely integrating custom logic, which can lead to conflicts with inherited behaviors. As a result, a series of security incidents, like the Audius attack [11], are caused by these conflicts. Moreover, the discussion on development framework issues indicates the lack of support for interacting and testing USCs from existing development frameworks.

To address these concerns, we suggest enhancing USC development frameworks with upgrade-aware development and testing environments. These concerns also motivate tooling to support behavioral regression testing on template functions and provide template-specific integration guides. These improvements would help developers safely customize and test USCs.

Security. Ensuring the security of USC is important for both developers and users. However, there are still uncovered security properties from two perspectives: 1) Insufficient checks on USC mechanism security. For example, the initialization issue is a main point in developers' discussions and can affect the reliability of USCs. Yet there is no tool for detecting such issues. Discovering such concerns can motivate the development of detectors for USC initialization flaws. 2) Insufficient checks on USC logic security. As shown in Table III, most of the reported USC security incidents are caused by logic flaws between different versions of the logic contracts, whereas current studies on USC security properties focus on USC mechanisms. Designing detection methods for high-level logic flaws is known to be difficult in many related studies [52], [82], [83], as the business logic of smart contracts is highly customized and diverse.

The newly identified concerns can motivate the research community to develop tools to improve USC security. For example, despite extensive studies have been proposed regarding access control security [51], [52], [54], the security incidents in RQ3 indicate a gap between practical tools for verifying access control policy and function compatibility before and after upgrades. Besides, a systematic risk assessment of the identified concerns can also guide the tooling for enhancing USC development. Possible methods include conducting developer surveys to evaluate the perceived severity and impact of each concern in actual deployment scenarios [16]. By collecting feedback from practitioners, tooling can prioritize concerns not

only by prevalence but also by their practical risks. We plan to include the risk assessment in our future work.

C. Threats to Validity

Internal Validity. The issues proposed in Section III are based on manual inspection after filtering the StackExchange posts with specific keywords. Thus, the internal threats to validity originate from two aspects: the coverage of the discussions and the accuracy of manual inspection.

Due to time and human resource limitations, it is not easy to cover all the keywords related to upgradeability. In our research, we iteratively select the keywords deriving from the upgradeable concept and implementation strategy. We also evaluated the representativeness of the selected keywords. There might be discussions that are uncovered by our research. Nonetheless, researchers can adopt our methods to more posts.

As the label and categorization of these discussions are defined and labeled by our manual analysis, the analysis result can be inherently subjective. To reduce inaccuracy in categorization, we followed the open card sorting design as in [16], [30] to categorize the posts based on the analysis results of two researchers. Both researchers have more than two years of experience in smart contract development. Moreover, we used Cohen's Kappa value to measure the agreement of the analysis results of the two researchers. The result shows strong agreement. For the process of identifying related posts, both researchers followed the same criterion and jointly determined if a post is related to USC.

Additionally, there can be overlaps between the identified concerns and existing smart contract studies and vulnerability categories. Some of the concerns, such as gas estimation and access control for USC pattern, can be treated as subproblems in general smart contracts, which is covered by existing studies. Still, the majority of the identified concerns originate from the specialized mechanisms of USC implementations and interactions, such as the Proxy-Logic Interaction concern.

External Validity. The source data in our study is collected from StackExchange posts, which may not precisely reflect the distribution of developers' concerns in real-world smart contract development. To mitigate this problem, we studied the real-world security impact of these concerns in Research Question 3. Future studies can complement our study by integrating other data sources, such as GitHub issues and developer surveys with a similar research design. The findings on new concerns can be extended. The findings on distribution of the concerns may vary with new data sources, as data sources such as GitHub issues inherently focus on different development stages compared with technical discussions. Meanwhile, the implementation of upgradeability is still under improvement. On the one hand, there are new EIPs for implementing upgradeability, such as EIP-6864 Upgradable Fungible Token, which was proposed in April 2023. On the other hand, the upgradeable smart contract templates provided by blockchain companies are updating frequently. The discussions about the new upgradeability implementations are growing quickly. The distribution and trend of the discussed topics can vary over time. Besides, these discussions can reflect new concerns, where the research design

in this study can be reused to identify these new problems and trends.

Construct Validity. The impact of the studied concerns is measured by their distribution in the studied posts. Due to the nature of the platform and its user base, the discussion on StackExchange can introduce preferences on concerns in the *design & development* phase of software SDLC, leading to bias in the analysis. To address this, we further investigated the real-world consequences of specific concerns, particularly those that may lead to code flaws, in RQ3. In future work, we aim to improve the unified measurement for assessing the impact of all categories of concerns by combining multiple sources, such as developer surveys.

VII. RELATED WORK

Empirical study on Smart Contract. The prevalence of smart contracts has motivated many empirical studies to demonstrate and facilitate developers' practice on smart contract software engineering. Zhou et al. [8] systematically studied the decentralized finance attacks and investigated potential defenses against them. Chaliasos et al. [56] proposed to evaluate the effectiveness of smart contract analyzers on decentralized finance attacks and then conduct a survey on practitioners to understand their perspective on security tools. Interviews and surveys are widely used for empirically studying developers' perceptions on smart contract security [84], development challenges [32], and usage of Ethereum functionalities [85], etc. Open card sorting has also been an widely adopted method to study bug categorization in the field of smart contracts [16], [30], [86] and wider software engineering [87], [88]. By mining StackExchange posts, Chen et al. [16] identified 20 main smart contract defects from aspects like security and reusability. Other studies have explored developers' difficulties in using cryptographic functions [33], smart contract libraries [17], compiler bugs [86], etc. In this work, we focus on the new trend of smart contract upgradeability. Based on open card sorting design and real-world incidents investigation, we propose to explore developers' concerns and problems in developing USCs, which exhibits distinct findings regarding USC.

Upgradeable Smart Contract Study. A series of studies have been conducted on the detection, characterization, and security analysis of USCs. USCHunt [2] and Mehdi et al. [89] summarized several USC patterns and proposed to detect USCs through source code analysis. USCDetector [?] extended the detection scope to smart contracts without source code and measured the prevalence of different implementation types. These works also proposed to evaluate several security properties of USCs, such as storage collision, function selector clash, and access control. Huang et al. [90] built a tool for detecting USCs from bytecode. They further investigated 300 random USC source codes and found that the reason for these upgrades mainly includes functionality change, vulnerability repair, and code optimization. Ruaro et al. [10] focused on the storage collision problem between proxy and logic smart contracts and proposed a detection method based on type inference and exploit generation. However, none of the existing work has

explored the developers' concerns in USC development. In this work, we study the practical concerns of developers and shed light on the unexplored perspectives that can enhance USC development.

VIII. CONCLUSION

In this work, we conduct the most comprehensive empirical study on exploring developers' concerns in upgradeability smart contract development. By mining 2,224 StackExchange posts, we identify 13 common development concerns across the smart contract development life cycle. Moreover, we study the prevalence of these concerns in discussions and find that the top 5 most concerned problems have not been covered by existing research, such as testing smart contracts under specific development frameworks. Furthermore, we evaluate the real-world impact of the identified concerns by analyzing security incidents and on-chain smart contracts. We find 26 related bugs in real-world smart contracts. Besides, we show that the causes of most upgradeable smart contract security incidents are related to the identified concerns.

REFERENCES

- [1] "Top blockchain DApps." DAppRadar. Accessed: Dec. 1, 2024. [Online]. Available: <https://dappradar.com/rankings>
- [2] W. E. Bodell III, S. Meisami, and Y. Duan, "Proxy hunting: Understanding and characterizing proxy-based upgradeable smart contracts in blockchains," in *Proc. 32nd USENIX Secur. Symp. (USENIX Security)*, 2023, pp. 1829–1846.
- [3] X. Li, J. Yang, J. Chen, Y. Tang, and X. Gao, "Characterizing Ethereum upgradeable smart contracts and their security implications," in *Proc. ACM Web Conf. (WWW)*, 2024, pp. 1847–1858.
- [4] A. M. Ebrahimi, B. Adams, G. A. Oliva, and A. E. Hassan, "A large-scale exploratory study on the proxy pattern in Ethereum," *Empirical Softw. Eng.*, vol. 29, no. 4, pp. 1–51, 2024.
- [5] S. Yang, J. Chen, M. Huang, Z. Zheng, and Y. Huang, "Uncover the premeditated attacks: Detecting exploitable reentrancy vulnerabilities by identifying attacker contracts," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2024, pp. 1–12, doi: 10.1145/3597503.3639153.
- [6] P. Bose, D. Das, Y. Chen, Y. Feng, C. Kruegel, and G. Vigna, "Sailfish: Vetting smart contract state-inconsistency bugs in seconds," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 161–178.
- [7] J. Zhang et al., "SmartReco: Detecting read-only reentrancy via fine-grained cross-DApp analysis," 2024. [Online]. Available: <https://arxiv.org/abs/2409.18468>
- [8] L. Zhou et al., "Sok: Decentralized finance (DeFi) attacks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 2444–2461.
- [9] X. Deng, S. M. Beillahi, C. Minwalla, H. Du, A. Veneris, and F. Long, "Safeguarding DeFi smart contracts against Oracle deviations," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2024, pp. 1–12, doi: 10.1145/3597503.3639225.
- [10] N. Ruaro, F. Gritti, R. McLaughlin, I. Grishchenko, C. Kruegel, and G. Vigna, "Not your type! Detecting storage collision vulnerabilities in ethereum smart contracts," in *Proc. Netw. Distrib. Syst. Secur. (NDSS) Symp.*, Feb. 2024, pp. 1–17.
- [11] "Audius governance takeover post." Audius. Accessed: Dec. 1, 2024. [Online]. Available: <https://blog.audius.co/article/audius-governance-takeover-post-mortem-7-23-22>
- [12] I. Qasse, M. Hamdaqa, and B. Ór Jónsson, "Smart contract upgradeability on the Ethereum blockchain platform: An exploratory study," 2023. [Online]. Available: <https://arxiv.org/abs/2304.06568>
- [13] M. Salehi, J. Clark, and M. Mannan, "Not so immutable: Upgradeability of smart contracts on Ethereum," in *Proc. Int. Conf. Financial Cryptography Data Secur.*, New York, NY, USA: Springer-Verlag, 2022, pp. 539–554.
- [14] P. K. Venkatesh, S. Wang, F. Zhang, Y. Zou, and A. E. Hassan, "What do client developers concern when using web APIs? An empirical study on developer forums and stack overflow," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2016, pp. 131–138.
- [15] D. Spencer, *Card Sorting: Designing Usable Categories*. New York, NY, USA: Rosenfeld Media, 2009.
- [16] J. Chen, X. Xia, D. Lo, J. Grundy, X. Luo, and T. Chen, "Defining smart contract defects on Ethereum," *IEEE Trans. Softw. Eng.*, vol. 48, no. 1, pp. 327–345, Jan. 2022.
- [17] M. Huang, J. Chen, Z. Jiang, and Z. Zheng, "Revealing hidden threats: An empirical study of library misuse in smart contracts," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2024, pp. 1–12, doi: 10.1145/3597503.3623335.
- [18] G. Wood et al., "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1–32, 2014.
- [19] M. A. Jorge Izquierdo, "Erc-897: Delegateproxy," 2018. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-897>
- [20] N. Mudge, "Erc-1538: Transparent contract standard," 2018. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-1538>
- [21] P. G. Gabriel Barros, "Erc-1822: Universal upgradeable proxy standard (UUPS)," 2019. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-1822>
- [22] H. C. Santiago Palladino and F. Giordano, "Erc-1967: Proxy storage slot," 2019. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-897>
- [23] N. Mudge, "Erc-2535: Diamonds, multi-facet proxy," 2020. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-2535>
- [24] A. Poller, L. Kocksch, S. Türpe, F. A. Epp, and K. Kinder-Kurlanda, "Can security become a routine? A study of organizational change in an agile software development group," in *Proc. ACM Conf. Comput. Supported Cooperative Work Social Comput. (CSCW)*, New York, NY, USA: ACM, 2017, pp. 2489–2503, doi: 10.1145/2998181.2998191.
- [25] M. Ren et al., "Making smart contract development more secure and easier," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2021, pp. 1360–1370, doi: 10.1145/3468264.3473929.
- [26] J. T. Liang, M. Arab, M. Ko, A. J. Ko, and T. D. LaToza, "A qualitative study on the implementation design decisions of developers," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 435–447, doi: 10.1109/ICSE48619.2023.00047.
- [27] J. Xie, H. R. Lipford, and B. Chu, "Why do programmers make security errors?" in *Proc. IEEE Symp. Vis. Languages Human-Centric Comput. (VL/HCC)*, 2011, pp. 161–164.
- [28] C. Richter, J. Haltermann, M.-C. Jakobs, F. Pauck, S. Schott, and H. Wehrheim, "Are neural bug detectors comparable to software developers on variable misuse bugs?" in *Proc. 37th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, New York, NY, USA: ACM, 2023, pp. 1–12, doi: 10.1145/3551349.3561156.
- [29] N. Kannengießer, S. Lins, C. Sander, K. Winter, H. Frey, and A. Sunyaev, "Challenges and common solutions in smart contract development," *IEEE Trans. Softw. Eng.*, vol. 48, no. 11, pp. 4291–4318, Nov. 2022.
- [30] S. Yang, J. Chen, and Z. Zheng, "Definition and detection of defects in NFT smart contracts," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, New York, NY, USA: ACM, 2023, pp. 373–384, doi: 10.1145/3597926.3598063.
- [31] "Stack exchange API." StackExchange. Accessed: Dec. 1, 2024. [Online]. Available: <https://api.stackexchange.com/>
- [32] W. Zou et al., "Smart contract development: Challenges and opportunities," *IEEE Trans. Softw. Eng.*, vol. 47, no. 10, pp. 2084–2106, Oct. 2021.
- [33] J. Zhang, J. Chen, Z. Wan, T. Chen, J. Gao, and Z. Chen, "When contracts meets crypto: Exploring developers' struggles with Ethereum cryptographic APIs," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2024, pp. 1–13, doi: 10.1145/3597503.3639131.
- [34] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, p. 37, 1960.
- [35] "Verifying proxy contracts on Etherscan." Ethereum StackExchange. Accessed: Dec. 1, 2024. [Online]. Available: <https://ethereum.stackexchange.com/questions/82919/verifying-proxy-contracts-on-etherscan>
- [36] H. Assal and S. Chiasson, "Security in the software development lifecycle," in *Proc. 14th Symp. Usable Privacy Secur. (SOUPS)*, 2018, pp. 281–296.
- [37] "Which is the best proxy contract?" Ethereum StackExchange. Accessed: Dec. 1, 2024. [Online]. Available: <https://ethereum.stackexchange.com/questions/140153>

- [38] "Ownable in upgradeable contracts." Ethereum StackExchange. Accessed: Jun. 1, 2024. [Online]. Available: <https://ethereum.stackexchange.com/questions/109323/ownable-in-upgradeable-contracts>
- [39] "Proxy contract delegatecall to pass a dynamic array to a function in the target contract." Ethereum StackExchange. Accessed: Dec. 1, 2024. [Online]. Available: <https://ethereum.stackexchange.com/questions/66276>
- [40] "Ethereum development environment for professionals." Hardhat. Accessed: Dec. 1, 2024. [Online]. Available: <https://hardhat.org/>
- [41] Foundry. "Foundry: A blazing fast, portable and modular toolkit for Ethereum application development written in rust." GitHub. Accessed: Dec. 1, 2024. [Online]. Available: <https://github.com/foundry-rs/foundry>
- [42] Truffle. "Truffle: A development environment, testing framework, and asset pipeline for Ethereum." GitHub. Accessed: Dec. 1, 2024. [Online]. Available: <https://github.com/trufflesuite/truffle>
- [43] Brownie. "Brownie: A Python-based development and testing framework for smart contracts targeting the Ethereum virtual machine." GitHub. Accessed: Dec. 1, 2024. [Online]. Available: <https://github.com/eth-brownie/brownie>
- [44] "Call fallback function from eth-brownie from proxy." Ethereum StackExchange. Accessed: Dec. 1, 2024. [Online]. Available: <https://ethereum.stackexchange.com/questions/100223>
- [45] "How much extra gas is incurred per transaction when using a proxy contract." Ethereum StackExchange. Accessed: Dec. 1, 2024. [Online]. Available: <https://ethereum.stackexchange.com/questions/117567>
- [46] "How to split smart contract for reducing size of smart contract." Ethereum StackExchange. [Online]. Available: <https://ethereum.stackexchange.com/questions/147064>
- [47] X. Chen, P. Liao, Y. Zhang, Y. Huang, and Z. Zheng, "Understanding code reuse in smart contracts," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 470–479.
- [48] K. Sun, Z. Xu, C. Liu, K. Li, and Y. Liu, "Demystifying the composition and code reuse in solidity smart contracts," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2023, pp. 796–807.
- [49] "What does a minimal erc-1967 proxy in vyper look like." Ethereum StackExchange. Accessed: Dec. 1, 2024. [Online]. Available: <https://ethereum.stackexchange.com/questions/149380/what-does-a-minimal-erc-1967-proxy-in-vyper-look-like>
- [50] "If contracts can be proxied to other contracts, how can a user trust them?" Ethereum StackExchange. Accessed: Jun. 1, 2024. [Online]. Available: <https://ethereum.stackexchange.com/questions/92574>
- [51] A. Ghaleb, J. Rubin, and K. Pattabiraman, "Achecker: Statically detecting smart contract access control vulnerabilities," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 945–956, doi: 10.1109/ICSE48619.2023.00087.
- [52] Y. Liu, Y. Li, S.-W. Lin, and C. Artho, "Finding permission bugs in smart contracts with role mining," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, New York, NY, USA: ACM, 2022, pp. 716–727, doi: 10.1145/3533767.3534372.
- [53] F. Gritti et al., "Confusum contractum: Confused deputy vulnerabilities in Ethereum smart contracts," in *Proc. 32nd USENIX Secur. Symp. (USENIX Security)*, 2023, pp. 1793–1810.
- [54] Z. Zhong, Z. Zheng, H.-N. Dai, Q. Xue, J. Chen, and Y. Nan, "PrettySmart: Detecting permission re-delegation vulnerability for token behaviors in smart contracts," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2024, pp. 2073–2084, doi: 10.1145/3597503.3639140.
- [55] "How to obtain the implementation address from an upgradeable proxy smart contract?" Ethereum StackExchange. Accessed: Dec. 1, 2024. [Online]. Available: <https://ethereum.stackexchange.com/questions/92384>
- [56] S. Chaliasos et al., "Smart contract and DeFi security tools: Do they meet the needs of practitioners?" in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2024, pp. 1–13, doi: 10.1145/3597503.3623302.
- [57] Z. Zheng, J. Su, J. Chen, D. Lo, Z. Zhong, and M. Ye, "DAppSCAN: Building large-scale datasets for smart contract weaknesses in DApp projects," *IEEE Trans. Softw. Eng.*, vol. 50, no. 6, pp. 1360–1373, Jun. 2024.
- [58] T. Chen et al., "GasChecker: Scalable analysis for discovering gas-inefficient smart contracts," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1433–1448, Jul./Sep. 2021.
- [59] A. Di Sorbo, S. Laudanna, A. Vacca, C. A. Visaggio, and G. Canfora, "Profiling gas consumption in solidity smart contracts," *J. Syst. Softw.*, vol. 186, Apr. 2022, Art. no. 111193.
- [60] J. Li, Z. Zhao, Z. Su, and W. Meng, "Gas-expensive patterns detection to optimize smart contracts," *Appl. Soft Comput.*, vol. 145, Sep. 2023, Art. no. 110542.
- [61] Z. Zhang, B. Zhang, W. Xu, and Z. Lin, "Demystifying exploitable bugs in smart contracts," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 615–627.
- [62] OpenZeppelin. "Merge OpenZeppelin 2.0 with initializer functions." GitHub. Accessed: Dec. 1, 2024. [Online]. Available: <https://github.com/OpenZeppelin/openzeppelin-contracts-upgradeable/pull/18>
- [63] "dYdX perpetual audit." OpenZeppelin. Accessed: Dec. 1, 2024. [Online]. Available: <https://blog.openzeppelin.com/dydx-perpetual-audit>
- [64] J. Su et al., "Defiwarder: Protecting DeFi apps from token leaking vulnerabilities," in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2023, pp. 1664–1675.
- [65] "Rekt news." Rekt. Accessed: Dec. 1, 2024. [Online]. Available: <https://rekt.news>
- [66] "SlowMist hacked database." SlowMist. Accessed: Dec. 1, 2024. [Online]. Available: <https://hacked.slowmist.io/>
- [67] "Compound errors - Rekt." Rekt. Accessed: Dec. 1, 2024. [Online]. Available: <https://rekt.news/compound-errors/>
- [68] "Dexible." Rekt. Accessed: Nov. 30, 2024. [Online]. Available: <https://rekt.news/dexible-rekt/>
- [69] "Safemoon." Rekt. Accessed: Nov. 30, 2024. [Online]. Available: <https://rekt.news/safemoon-rekt/>
- [70] "Deus dao - Rekt." Rekt. Accessed: Nov. 30, 2024. [Online]. Available: <https://rekt.news/deus-dao-r3kt/>
- [71] "Level finance - Rekt." Rekt. Accessed: Dec. 1, 2024. [Online]. Available: <https://rekt.news/level-finance-rekt/>
- [72] "Astaria vulnerbaility." Astaria. Accessed: Dec. 1, 2024. [Online]. Available: <https://x.com/AstariaXYZ/status/1671215491069927429>
- [73] "OKX DEX - Rekt." Rekt. Accessed: Dec. 1, 2024. [Online]. Available: <https://rekt.news/okx-dex-rekt/>
- [74] "Unizen - Rekt." Rekt. Accessed: Nov. 30, 2024. [Online]. Available: <https://rekt.news/unizen-rekt/>
- [75] "How was NGFS token exploited?" Neptune Mutual. Accessed: Dec. 1, 2024. [Online]. Available: <https://neptunemutual.com/blog/how-was-ngfs-token-exploited/>
- [76] "Update on the exploit and the next steps." Perpy-Finance. Accessed: Dec. 1, 2024. [Online]. Available: <https://perpy-finance.beehiiv.com/p/update-recent-exploit>
- [77] Y. Sun et al., "GptScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2024, pp. 1–13, doi: 10.1145/3597503.3639117.
- [78] Y. Duan et al., "Towards automated safety vetting of smart contracts in decentralized applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, New York, NY, USA: ACM, 2022, pp. 921–935, doi: 10.1145/3548606.3559384.
- [79] A. Pernenev, D. Dimitrov, P. Tsankov, D. Drachsler-Cohen, and M. Vechev, "VerX: Safety verification of smart contracts," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2020, pp. 1661–1677.
- [80] S. Yang et al., "Hyperion: Unveiling DApp inconsistencies using LLM and dataflow-guided symbolic execution," 2024. [Online]. Available: <https://arxiv.org/abs/2408.06037>
- [81] "Remix release: Support for upgradeable contracts." GitHub. Accessed: Dec. 1, 2024. [Online]. Available: <https://github.com/ethereum/remix-project/releases/tag/v0.25.0>
- [82] M. Ye, Y. Nan, Z. Zheng, D. Wu, and H. Li, "Detecting state inconsistency bugs in DApps via on-chain transaction replay and fuzzing," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, New York, NY, USA: ACM, 2023, pp. 298–309, doi: 10.1145/3597926.3598057.
- [83] B. Zhang, "Towards finding accounting errors in smart contracts," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2024, pp. 1–13, doi: 10.1145/3597503.3639128.
- [84] Z. Wan, X. Xia, D. Lo, J. Chen, X. Luo, and X. Yang, "Smart contract security: A practitioners' perspective: The artifact of a paper accepted in the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)," in *Proc. 43rd Int. Conf. Softw. Eng., Companion (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 227–228, doi: 10.1109/ICSE-Companion52605.2021.00104.
- [85] J. Chen, X. Xia, D. Lo, and J. Grundy, "Why do smart contracts self-destruct? Investigating the selfdestruct function on Ethereum," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 2, pp. 1–37, Dec. 2021, doi: 10.1145/3488245.

- [86] H. Ma, W. Zhang, Q. Shen, Y. Tian, J. Chen, and S.-C. Cheung, "Towards understanding the bugs in solidity compiler," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA)*, New York, NY, USA: ACM, 2024, pp. 1312–1324, doi: 10.1145/3650212.3680362.
- [87] Q. Shen, H. Ma, J. Chen, Y. Tian, S.-C. Cheung, and X. Chen, "A comprehensive study of deep learning compiler bugs," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2021, pp. 968–980, doi: 10.1145/3468264.3468591.
- [88] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan, "A comprehensive study on deep learning bug characteristics," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2019, pp. 510–520, doi: 10.1145/3338906.3338955.
- [89] M. Salehi, J. Clark, and M. Mannan, "Not so immutable: Upgradeability of smart contracts on Ethereum," in *Proc. Int. Workshops Financial Cryptography Data Secur. (FC)*, S. Matsuo et al., Eds. Cham, Switzerland: Springer International Publishing, 2023, pp. 539–554.
- [90] Y. Huang et al., "The sword of Damocles: Upgradeable smart contract in Ethereum," in *Proc. 32nd IEEE/ACM Int. Conf. Program Comprehension (ICPC)*, New York, NY, USA: ACM, 2024, pp. 333–345, doi: 10.1145/3643916.3644426.